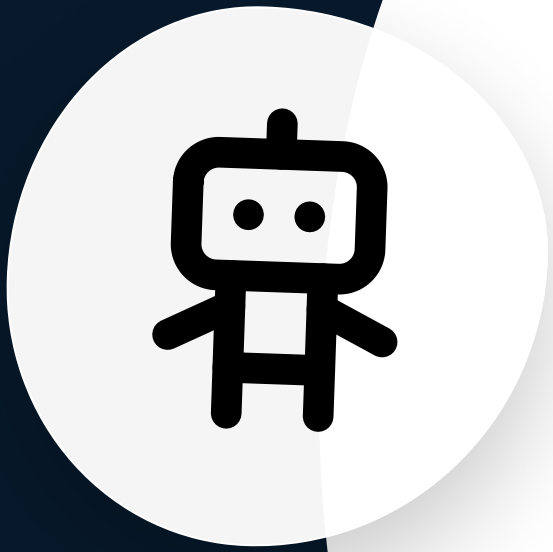
Agentes

M6: v0.4, Agentes



Hoy: el recorrido

De pasos fijos a un modelo que elige



Venís de v0.3: siempre busca y responde. Hoy le das herramientas y lo dejás elegir.

Sin mirar: ¿qué hace siempre tu v0.3?



?

¿Qué pasos?

sigue tu v0.3, siempre en el mismo orden



?

¿Otro camino?

una pregunta tuya que necesite la web o una cuenta

Respondé en una línea, sin mirar.

¿Qué hace siempre tu `v0.3` ?

Respuestas: pasos fijos o elige el modelo



Siempre igual

buscar, armar el contexto, responder



Otro camino

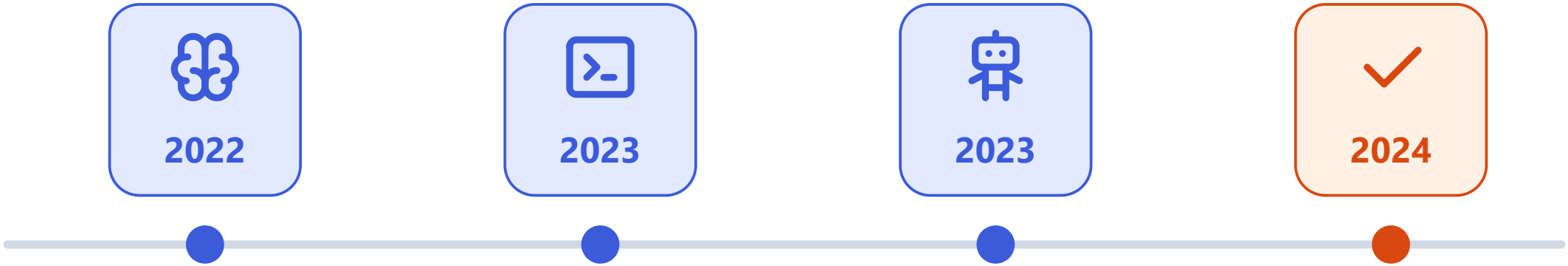
algo reciente (la web) o una cuenta exacta



Hoy

que el modelo elija el camino: eso es un agente

Del texto a pedir herramientas



La lección: usá un agente solo si hace falta.

Workflow: pasos fijos. Agente: elige él



Workflow

el camino lo decidís vos · costo predecible · conviene si conocés los pasos

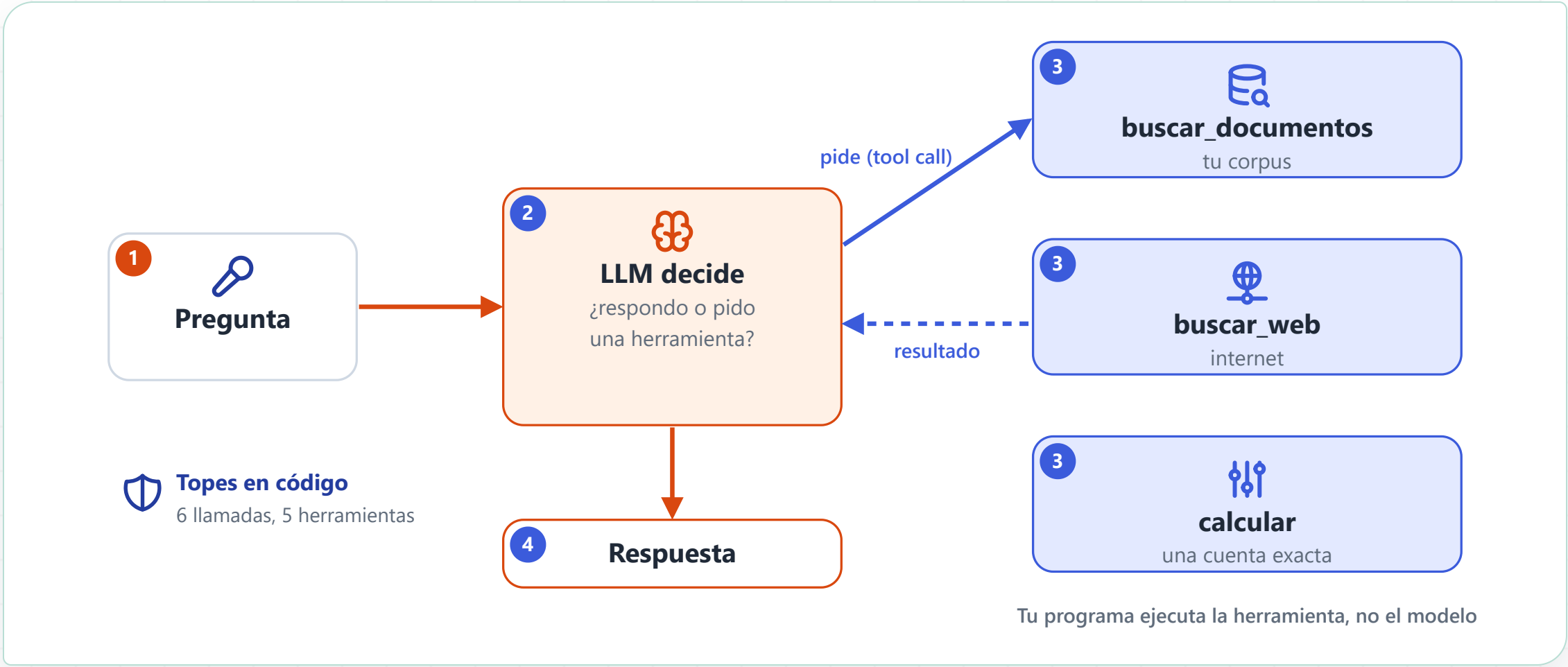


Agente

el camino lo decide el modelo · costo variable · conviene si los pasos cambian

Una receta, o un cocinero que mira la heladera y decide.

Un agente es un LLM dentro de un ciclo



Termina cuando responde o cuando llega al tope.

Las piezas de un agente, con su nombre

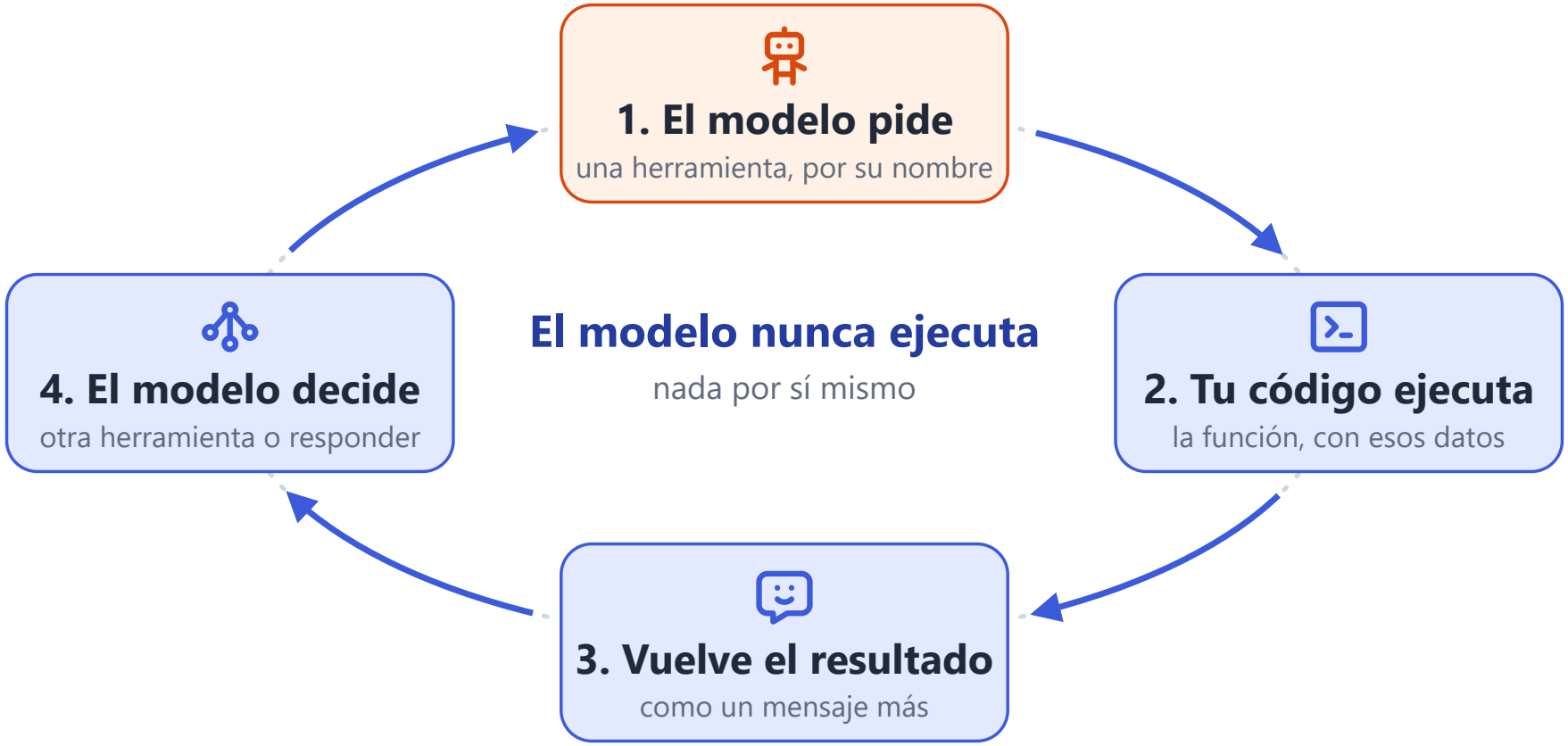
Pieza	Qué es
Herramienta (*tool*)	Función que el modelo pide
Pedido (*tool call*)	Nombre y datos; lo corre tu código
Ciclo (*ReAct*)	Piensa, pide, mira y repite
Tope	Cuántas llamadas, como máximo
Instrucciones	Cuándo usar cada herramienta

Para armar agentes hay varios kits

- LangChain create_agent
- Google ADK
- OpenAI Agents SDK
- Claude Agent SDK
- CrewAI
- smolagents (Hugging Face)
- PydanticAI

Todos hacen el mismo ciclo: el modelo pide una herramienta, tu código la corre. `AgentExecutor` era la forma vieja de LangChain; desde la v1, `create_agent`.

El modelo pide, tu código ejecuta



Ese ciclo se llama **ReAct**.

Así se ve el ciclo de tu v0.4, mensaje por mensaje

```
[pregunta]     …¿cuántas filas tiene la tabla que compara checkpointer y store?
               Multiplicá ese número por 3.
[modelo pide]  buscar_documentos({'consulta': 'checkpointer store tabla comparar…'})
[tu código]    buscar_documentos → '[langgraph__persistence] ## Checkpointer vs…'
[modelo pide]  calcular({'b': 3, 'operacion': 'multiplicacion', 'a': 5})
[tu código]    calcular → '15'
[respuesta]    La tabla … tiene 5 filas … el resultado es **15**.
```

- Dos herramientas, tres llamadas: dos piden, la tercera responde.
- Gemini 3.5 Flash-Lite. Otra corrida contó la fila de enlaces: 6 × 3 = 18.

El modelo solo ve nombre, tipos y descripción



Nombre

que no se confunda con otro



Descripción

qué hace y cuándo usarla



Errores

que dicen qué hacer si falla



Tipos

número, texto u opciones cerradas

La descripción vaga no cambió la elección

Alguien dice cuándo
12 de 12: descripción real, o vaga con pistas

=

Nadie dice cuándo
«Busca información.», SISTEMA neutro
11 y 12 de 12

gpt-5.4-nano, dos corridas: azar.

Una herramienta es una función con etiqueta

```python
@tool
def buscar_documentos(consulta: str) -> str:
    """Busca en mis documentos sobre LangGraph y LangChain."""
    return "\n\n".join(f"[{h['doc']}]\n{h['texto'][:600]}"
                       for h in buscar(consulta))
```

Como la descripción de un campo de Power BI: la lee para elegir.

Ejecutar código: poderoso y riesgoso

La calculadora

lo que usa tu v0.4: solo hace cuentas, con datos que tu código valida

Ejecutar código

el modelo escribe Python y se corre: analiza datos y grafica, pero puede hacer cualquier cosa

Ejecutar código pide un entorno aislado y que alguien apruebe.

Tu agente de código lo hace: por eso pide permiso.

Los límites van en el código, no en el prompt

Tope de llamadas

por consulta

Solo lectura

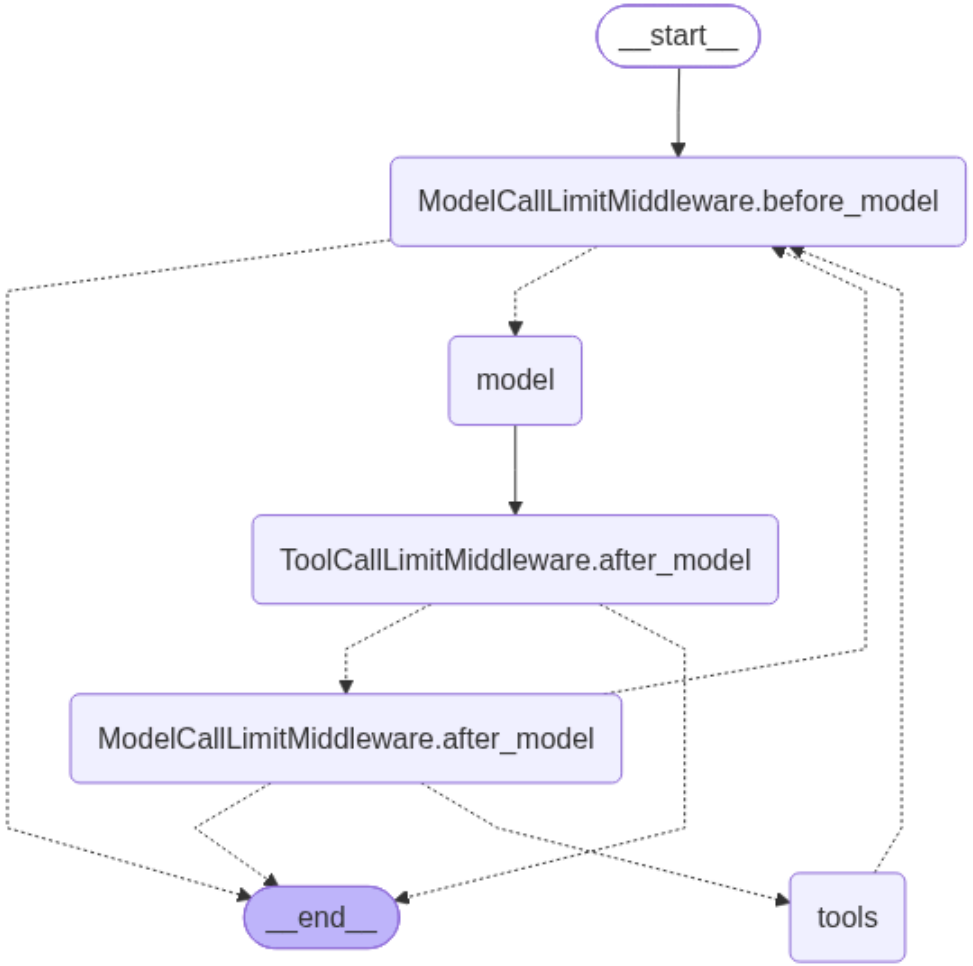
herramientas que no cambian nada

Cálculo fijo

el modelo no escribe ni corre código

Todo se configura en create_agent, no se pide por escrito.

En el grafo real, los topes son cajas del ciclo



Con la web caída, uno inventó y otro avisó

Modelo local chico
«La última versión es 0.1.2»

≠

Gemini 3.5 Flash-Lite (17/10)
«devolvió un error»

- ¿Cuál liberarías?
- ¿Qué haría falta para que el chico avise?
- Pista: ¿pidió la web?

Con la web caída, el ERROR llegó al modelo y lo dijo

```
[pregunta]     ¿Cuál es la última versión de LangGraph publicada en PyPI?
[modelo pide]  buscar_web({'consulta': 'langgraph pypi version'})
[tu código]    buscar_web → 'ERROR: la búsqueda web no respondió. No inventes; …'
[respuesta]    Lo siento, la búsqueda web devolvió un error, por lo que no puedo
               confirmar la última versión exacta de LangGraph…
```

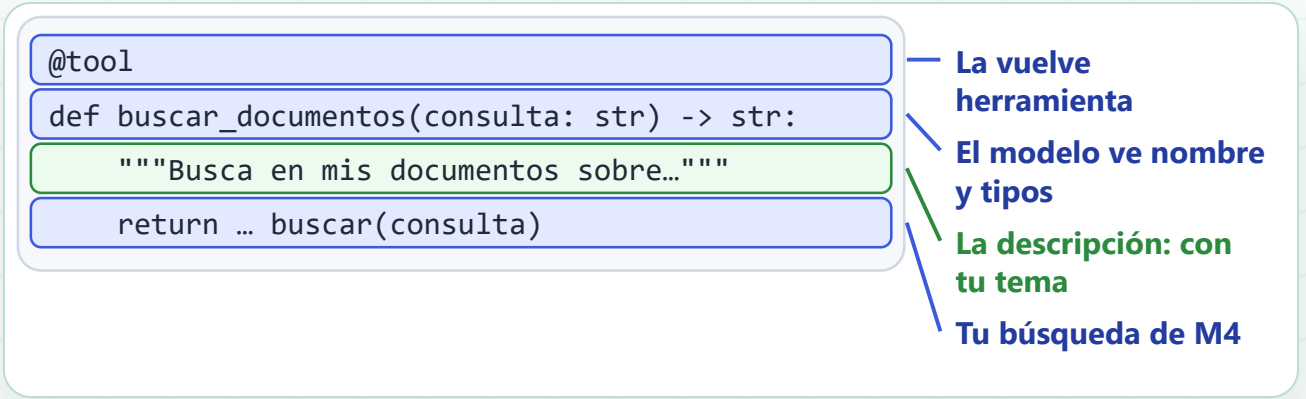
- Gemini 3.5 Flash-Lite leyó el ERROR.
- Si lo ignora, lo ataja el guardrail (clase 7). Si ni pide la web, no.

Taller: tres herramientas para tu tema (55 min)

1. **Tu descripción** de `buscar_documentos`.
2. **Tu suite de selección:** `datos/mi_seleccion.jsonl`
3. **Predecí** dónde falla.

Práctica M6, arranque y «En clase», paso 1.

Primer control · 10–15 min: ves qué herramienta pidió en cada pregunta.



Mismas descripciones, otro modelo: de 9 a 12 de 12

Modelo local chico
eligió bien 9 de 12

→

Gemini 3.5 Flash-Lite
12 de 12, en 3 corridas

- Misma suite, 17/10.
- Antes de culpar a la descripción, probá otro modelo.

¿Por qué falló la pregunta 12?

Esperaba

buscar_documentos y calcular

Hizo

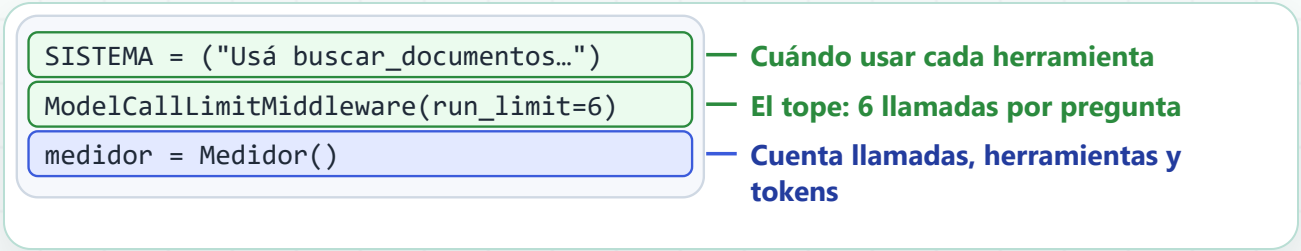
solo la primera; contó columnas (2), no filas (5): dio 6

¿Qué cambiarías?

una sola cosa

Taller: medir la elección y romper la web (50 min)

1. **Medí** con `web = "grabada"`
2. **Una** cosa y repetí.
3. **`web = "caida"`:** ¿avisa o inventa? Después, `"real"`



Práctica M6, «En clase», pasos 2 a 6.

Trampa: si ni pidió `buscar_web`, falló al elegir, no la web.

Repaso: hoy te llevás tres ideas



Agente

el modelo elige el camino



Herramienta

tu código la ejecuta



Topes

en el código, no en el prompt

El modelo elige por el nombre y la descripción de cada herramienta.

¿Por qué eligió mal esta vez?

Mostrá una pregunta donde eligió mal la herramienta.

¿Fue la descripción, las instrucciones, la pregunta o el modelo?

Anotá tu **confianza (1 a 4)**. Entrega de avance 3: `evidencia/v0.3.md`, `evidencia/v0.4.md` y *push* hasta el mié 18/11, 23:59.